Health Data Mapping, Insight Discovery and AI Readiness

Abdulrahman Alazhari

AI Readiness and Insight Discovery

As the Ikarians platform transitions from raw data ingestion to advanced machine learning integration, it is critical to construct a framework that optimises data for Large Language Models (LLMs) while maintaining clinical safety. This report outlines establishing an AI Readiness Framework, optimising the Health Summary Structure, prioritising compound clinical signals over algorithmic noise, and possible new insights to ensure the Ikarus AI delivers precise, actionable, and safe health insights.

I. AI Readiness Framework

To operate effectively in a healthcare context without hallucinating, the Ikarus AI cannot rely solely on generalised internet training. It should operate within a structured, multi-layered framework.

A. Guardrail Agent System:

Rather than relying on a single prompt instructing the AI not to prescribe or diagnose, the architecture should utilise specialised, secondary AI agents. These act as an automated compliance filter, analysing the primary AI's output in real-time and blocking any text that attempts to offer unauthorized medical advice. There are other critical guardrails we can implement alongside the secondary AI agent. These additional guardrails can use logic rules and pharmacology data, to ensure patient safety as follows:

1. AI-Ready "If/Then" Safety Rules

We can build robust, pharmacy-driven data pipelines and logic rules that act as clinical guardrails. These prevent the AI from generating nudges that might actually be dangerous given a patient's specific genetic or pharmacological profile. Examples include:

- **Drug-Diet Interactions:**

  IF a patient is prescribed warfarin, THEN strictly limit nudges for green, leafy diets (which are high in Vitamin K and antagonise the medication), and alert the clinician to monitor the patient's INR.

  IF a patient is prescribed tetracycline or quinolone antibiotics (such as ciprofloxacin), THEN strictly limit nudges for calcium-rich foods like milk, yogurt, and cheese around the time of their dosage and alert the clinician to advise the patient to separate dairy consumption from the medication by at least 1 to 2 hours.

- **Disease-Diet Conflicts:**

  IF a patient has celiac disease, THEN block all nudges suggesting wheat-based complex carbohydrates.

  IF a patient is diagnosed with gout, THEN strictly limit nudges for red meat, organ meats, certain seafoods (like salmon) and alert the clinician to monitor the patient for gout flares.

2. Optimised Timing Nudges

Guardrails can also be applied when the AI is allowed to interact with the patient, ensuring behavioral nudges actively support proper medication use.

For example: Timing physical activity nudges so they explicitly do not coincide with the peak effects of antihypertensive drugs, preventing sudden drops in blood pressure.

B. Structured Retrieval-Augmented Generation (RAG)

The AI's responses should be anchored in the patient's actual data. The ETL (Extract, Transform, Load) pipelines serve as the objective foundation for this Grounding layer. By unifying fragmented data from Apple, Garmin, Google, and Samsung into standard ik_variables, the AI is fed clean, interoperable data.

RAG (Retrieval-Augmented Generation) should also pull from the patient's subjective inputs to provide holistic context. This includes data entered manually into the patient's daily health journal, self-reported survey scores, and critical information the AI picks up during chat conversations.

The system can retrieve not just chat logs, but hundreds of past clinical notes, multi-omics data, and historic wearable trends across the patient's entire disease journey. By anchoring the AI in this deep, longitudinal data, the system can answer complex questions about the patient's health and linking possible causes together.

C. Grounding Data and Tools Integration

The framework should allow the AI to actively search external APIs or specific internal sources to verify information on demand.

While the Grounding layer provides a foundation by combining the patient's historical profile with Ikarians' pre-approved internal medical document, it cannot contain everything. The AI cannot natively store all of the world's continuously updating medical knowledge (such as live external drug databases) without risking hallucinations or exceeding its token limits.

Tool integration enables the AI system to conditionally query external APIs, retrieving precise, continuously updated clinical data on an as-needed basis during user interactions. Examples of useful tools that maybe considered are Pharmacovigilance APIs, Computer Vision Dietary Extractors and Validation Checkers.

II - Proposed Health Summary Structure

To ensure the primary AI agent provides accurate, hallucination-free insights, the system cannot feed it raw, minute-by-minute wearable data. Instead, the architecture should utilise specific summarisation methods to optimise the AI's context window, ensuring it only receives clean, high-density clinical signals. This can be achieved by:

1. Token Reduction via Algorithmic Aggregation The system should pre-summarise thousands of high-frequency data points from wearable APIs before they ever reach the AI's context window.

- **Cumulative Aggregation:** For continuously accumulating metrics (like daily steps or active energy burned), the pipeline should summarise the data using a daily **sum**.
- **Discrete Arithmetic Aggregation:** For non-cumulative physiological metrics (like heart rate or body temperature), the pipeline should summarise the data points using statistics like the **simple arithmetic mean, minimum, or maximum** values. This translates a chaotic stream of raw heartbeats into a single, highly optimized daily token (e.g. "Average Resting HR: 60 bpm").

2. Chronological Alignment via Standard Time-Windows

Because different wearable brands send data at varying frequencies, the summarisation pipeline should apply a Standard Time-Window transformation. By grouping all intermittent measurements by patient ID and date, the system summarises the data into a single 24-hour integer (e.g. total daily sleep hours). This ensures the AI reads a standardised, chronological summary rather than becoming confused by disjointed timestamp formats.

3. Computational Efficiency via Data Fusion

To save computational power and improve insight accuracy, the system should summarise low-level features by fusing them together into high-level compound signals before feeding them into the AI.

Example: Instead of feeding the AI raw logs of brief sleep awakenings (which is often just normal sleep architecture noise), the system fuses a sharp drop in Oxygen Saturation with a simultaneous spike in Respiratory Rate. The AI's context window is simply fed the summarised clinical alert: "High Risk: Obstructive Sleep Apnea event detected," saving it from having to deduce the condition from raw tables.

4. Programmatic Statistical Pre-Processing

For longitudinal patient tracking, the platform should utilise statistical programming (such as R) to pre-process the patient's entire historical dataset.

- Instead of uploading hundreds of past clinical rows into the prompt, functions like summary(), mean(), or group_by() can be used to generate polished summary tables and quantitative summaries.
- The AI delivers a highly optimised string of text, such as: "Over the last 90 days, the patient's mean blood glucose was X, with a maximum of Y."

By aggregating, time-windowing, and fusing the data prior to AI ingestion, the grounding data remains small and signal-dense. This allows the AI agent to utilise its context window for its actual purpose rather than wasting tokens processing raw math.

III - Recommendations for Data Prioritisation

To further protect computational context, the platform should prioritise data that dictates true biological changes over temporary daily fluctuations.

1. Prioritising Compound Signals

The system should prioritise instances where objective wearable metrics intersect with subjective patient-reported data and prioritise this as a signal. For example, if a chronically elevated wearable stress baseline intersects with an abnormally high T-score on a scheduled **PROMIS Social Isolation** or **PROMIS Global Mental Health** assessment, the AI categorises this as a high-priority "Chronic Psychosocial Stress" signal.

2. Deprioritising Algorithmic Noise

The AI should be programmed to deprioritise noises of raw cross-brand comparisons and instead, calculates the patient's individual Z-score to track long-term rates of change against their own specific baseline.

IV - Suggested Approach for AI-Ready Inputs

To improve the AI's context quality, the architecture should focus on feeding the agent only highly relevant, signal-dense clinical data.

Here is a review of the key opportunities to optimise and improve the AI's context quality:

1. Standardised Data Schemas

Before data enters the AI's context window, it should be mapped to a standardised schema. Because patients use varying wearable brands, the AI should never receive brand-specific variable names. All objective inputs should be converted into Ikarians Standard Variables (e.g. ik_steps, ik_distance_meters) with unified units and data types. This ensures the AI interprets a consistent language regardless of the hardware source.

2. Injecting Summarised & Fused Metrics

As mentioned, we can inject data as clean, aggregated variables: Cumulative variables (steps, calories) are inputted as daily sums. Discrete variables (heart rate, body temperature) are inputted as summary statistics (mean, min, max). Subjective variables are inputted as standardized PROMIS T-scores (e.g. "PROMIS Social Isolation T-Score: 60") rather than raw survey answers.

3. Strategic Prompt Architecture Mapping

AI-ready inputs should be segregated into the agent's distinct operational layers to maintain clinical guardrails:

The System Prompt: This layer receives the hardcoded clinical rules and guardrails (e.g. "You are a health coach. Do not diagnose or prescribe").

The Grounding Data: This layer receives the pre-processed, summarised wearable metrics and the specific internal clinical documentation the AI is allowed to reference.

The User Prompt: This layer receives the immediate, dynamic chat message from the patient (e.g."How did I sleep this week?").

4. Dynamic Data Injection via RAG (Retrieval-Augmented Generation)

For historical inputs, the architecture should use RAG to perform recursive vector searches. Instead of loading the entire profile, the system stores historical data as separate chunks that can be retrieved when needed to give responses.

5. AI Tools Integration

As already mentioned, to prevent the AI from exceeding its token limits or hallucinating clinical facts, the AI can execute a targeted internal tool (a specifically designed function with strict inputs and outputs).

The Ikarus server uses that tool to query a massive external API on the backend, and only the precise, narrow result of that query is injected back into the prompt as a new AI-Ready Input.

Examples of this approach include designing specific tools that query Pharmacovigilance APIs, Computer Vision Dietary Extractors, and Data Validation Checkers as follows.

V- Insight Discovery for AI Tools

Proposed AI Tools:

1. The Signal vs. Noise Validator

- **NAME:** filter_signal_from_noise
- **ONE-LINER:** Evaluates a sudden physiological spike against the patient's baseline to detect extreme deviations (>3 S.D.) and stops standard AI nudges to prevent inappropriate advice during a potential medical crisis.
- **INPUTS:** patient_id (STRING), metric_name (STRING), current_value (FLOAT)

- **RETURNS:** A JSON object containing the Z-score and an emergency escalation protocol. Example: {“metric”: “resting_heart_rate”, “z_score”: 3.8, “status”: “SEVERE_OUTLIER_DETECTED”, “action_required”: “Stop lifestyle nudges; ask patient to verify symptoms; flag for critical_patients dashboard”, “missing_data”: “None”}
- **WHY IT HELPS:** The platform focuses on longevity and daily habits, but patients can still experience sudden medical emergencies. While manually capping acceptable values (e.g. setting a universal maximum limit for heart rate or sleep duration) acts as a basic safety net, it is less accurate because it ignores individual patient context. Comparing metrics against a patient’s own historical baseline is far superior, as it dynamically accounts for the patient’s unique health status, comorbidities, and existing risk factors. A heart rate that is “normal” for an athlete could indicate a severe crisis for a patient with cardiovascular disease. If a patient’s heart rate spikes 3.5 standard deviations above their own specific baseline, this tool catches the severe outlier before a draft is sent. It forces the AI to ask the patient to confirm if they feel unwell, and subsequently alerting the real physician through the physician dashboard.
- **DIRECTION:** Improving AI Validation.
- **RISKS:** The primary risk is a hardware glitch (e.g. a loose smartwatch strap recording a false heart rate outlier measurement) triggering unnecessary panic. The design keeps it safe by strictly enforcing the Read-Only and Walled Garden rules. The tool never automatically dials emergency services or gives a diagnosis. Instead, it simply asks the patient to verify their health status and flags the anomaly on the doctor’s dashboard, ensuring the AI catches its contextual mistake without overstepping its clinical boundaries.

2. The Biometric-Pharmacological Cross-Referencer

- **NAME:** verify_biometric_drug_interactions
- **ONE-LINER:** Cross-references a patient’s active medication list (automatically extracted and standardised into ATC codes when the patient scans their e-prescriptions or medication boxes) against anomalous wearable biometric data to check if a physiological spike is a drug side-effect rather than a lifestyle issue.
- **INPUTS:** patient_id (STRING), scanned_atc_code_list (LIST of STRINGS), wearable_metric (STRING), current_value (FLOAT).
- **RETURNS:** A JSON object detailing the verified pharmacological interaction (retrieved via cross-referencing an external medication database such as Lexicomp or Micromedex), any compound risks, and the required AI action. Example: {“anomaly”: “ecg_signal irregularity / prolonged intervals”, “possible_drug_cause”: “Citalopram and Azithromycin”, “database_source”: “Lexicomp”, “compound_risk”: “Concurrent use of potassium-depleting loop diuretic”, “status”: “VERIFIED_QT_PROLONGATION_RISK”, “action”: “Block stress management nudges; Flag for urgent physician review for potential torsade de pointes”, “missing_data”: “None”}
- **WHY IT HELPS:** A purely behavioral AI can easily misinterpret dangerous drug side effects as lifestyle trends. Furthermore, because patients often fail to manually report their exact pharmacological profiles accurately, allowing the patient to simply scan their electronic prescriptions or physical medication packages for the ATC code (a recognised indexing and classification system coding for medications, created and implemented by the World Health Organization) would ensure accurate medication record. This intercepts AI errors in critical scenario, for example: ECG Anomalies (QT Prolongation & Torsade de Pointes): If the wearable detects an irregular heartbeat, purely the AI might suggest stress management or flag generalised arrhythmia. This tool checks the scanned ATC codes for high-risk drugs that cause QT-interval prolongation, leading to life-threatening arrhythmias (torsade de pointes). The same applies for other established side effects that can be accessed through an external medication database (e.g Lexicomp or Micromedex) and are linked to the wearables measured metrics.

- **DIRECTION:** Improving AI Validation.
- **RISKS:** The camera scanner might misread a blurry medication label, or the patient might scan an over-the-counter supplement that lacks a standard ATC code, causing the AI to miss the pharmacological link. The design keeps it safe because it is strictly a read-only safety net. If it finds no ATC correlation, it simply defaults back to standard monitoring. It never instructs the patient to stop taking their medication and does not attempt to independently diagnose an illness.

3. The Nutrition Facts Label & Portion Scanner

- **NAME:** scan_nutrition_table_and_portion_image
- **ONE-LINER:** Scans a food product's Nutrition Facts table to extract exact nutrients, while simultaneously analysing a photo of the patient's physical plate to estimate the exact portion consumed, automating the entire logging process.
- **INPUTS:** patient_id (STRING), nutrition_label_image_url (STRING), plated_food_image_url (STRING)
- **RETURNS:** A JSON object containing the exact nutritional data extracted from the label, the visually estimated portion size from the plate image, and the final calculated intake. Example: {"label_per_serving": {"calories": 150, "sodium": "210mg", "trans_fats": "0g"}, "estimated_portion_eaten": "1.5 servings", "total_consumed_estimate": {"calories": 225, "sodium": "315mg"}, "confidence_score": "85%", "missing_data": "None"}
- **WHY IT HELPS:** Patients face significant survey fatigue and a high respondent burden when asked to manually calculate portion sizes and type out complex data from nutrition labels. In addition, identifying specific dietary risks such as hidden added sugars, industrial trans fats, or excessive sodium is difficult for the average patient. By using Optical Character Recognition (OCR) to read the exact table of calories/contents and computer vision to estimate the physical portion volume on the plate, this tool removes the exhausting manual data entry for the patient while giving the physician highly precise, objective tracking data.
- **DIRECTION:** Improving AI Capability
- **RISKS:** Computer vision can struggle with depth perception for portion sizes, or the label image might be blurry and misread. The design keeps it safe by treating the calculated intake as an estimate with a stated "confidence score" rather than a hard clinical diagnostic. To mitigate depth perception risks, the AI can be programmed to casually ask the user to include a standard reference object (like a fork or their hand) in the photo to help the tool scale the portions accurately.

Conclusion

By implementing this AI Readiness Framework, Ikarians ensures that its complex, multi-source health data is transformed into structured, actionable insights. Filtering algorithmic noise through subjective PROMIS tools and strictly managing the AI's context window will safeguard the platform's clinical integrity, allowing Ikarus AI to accurately monitor and improve the six pillars of patient longevity.

References:

- Gao Y, Xiong Y, Gao X, et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*. Published online 2023. doi:10.48550/arXiv.2312.10997
- Gandara N, Ordax E, Daggumalli S, Dubey A. *Generative AI at AWS*. Packt Publishing; 2024.